Azure Service Bus – Cross-Entity Transactions

1. What is a Cross-Entity Transaction?

A **cross-entity transaction** in Azure Service Bus allows us to perform operations on **multiple queues or topics as one atomic transaction**.

For example, suppose we have three queues:

```
Q1 → Source Queue
Q2 → Destination Queue
Q3 → Destination Queue
```

Our requirement is:

1. Read a message from Q1.
2. Write the message to Q2.
3. Write the same message to Q3.
4. Delete/complete the message from Q1.
5. All these operations should succeed together.

The important requirement is:

> The message should be removed from Q1 only if writing to both Q2 and Q3 succeeds.

---

2. Why Do We Need a Transaction?

Without a transaction, imagine this situation:

```
Q1 → Read Message
       ↓
     Write to Q2 ✓
       ↓
     Write to Q3 ✗
       ↓
     Complete Q1
```

If we complete the message from Q1 even though Q3 failed, we can lose the message.

The message would be:

```
Q1 → Deleted
Q2 → Message exists
Q3 → Message does not exist
```

This creates an inconsistent state.

With a transaction:

```
Q1 → Read
 ↓
Q2 → Write ✓
 ↓
Q3 → Write ✓
 ↓
Q1 → Complete ✓
 ↓
Transaction Commit
```

If any operation fails:

```
Q1 → Read
 ↓
Q2 → Write ✓
 ↓
Q3 → Write ✗
 ↓
Transaction Rollback
```

The transaction ensures that the operations are treated as one unit.

---

3. Real-Time Example

Suppose Q1 contains:

```
Hello Transaction
```

Initially:

```
Q1 → 1 message

Q2 → 0 messages

Q3 → 0 messages
```

We want to achieve:

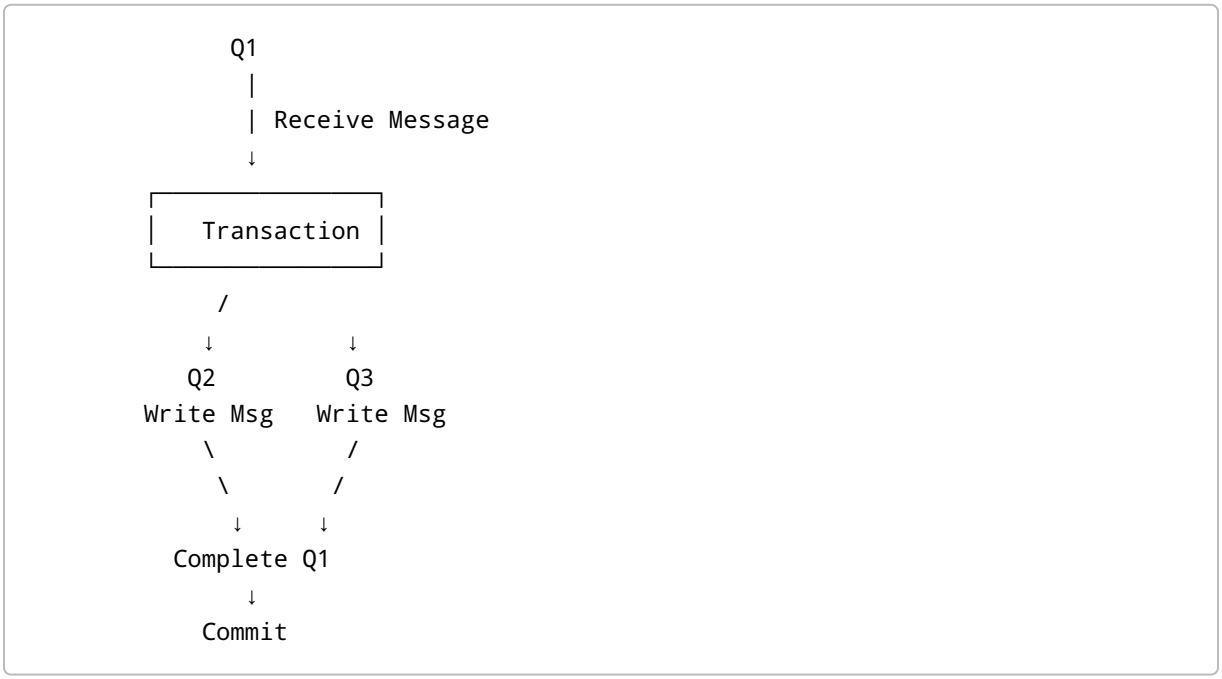
```
Q1 → 0 messages

Q2 → 1 message

Q3 → 1 message
```

The message should move from Q1 to both Q2 and Q3 as part of one transaction.

---

4. Transaction Flow

The complete flow is:



The important point is that **Q1 is completed only after the writes to Q2 and Q3 are successful**.

---

5. Namespace-Level Connection String

For cross-entity transactions, the queues involved in the transaction must belong to the **same Service Bus namespace**.

For example:

```
Service Bus Namespace
        |
        ├── Q1
        ├── Q2
        └── Q3
```

The connection is made at the namespace level so that the application can access all three entities.

---

6. Enable Cross-Entity Transactions

When creating the `ServiceBusClient`, cross-entity transactions need to be enabled.

Example:

```
var clientOptions = new ServiceBusClientOptions
{
    EnableCrossEntityTransactions = true
};

ServiceBusClient client =
    new ServiceBusClient(connectionString, clientOptions);
```

The important property is:

```
EnableCrossEntityTransactions = true
```

This allows transaction operations to span multiple Service Bus entities.

---

7. Create Receiver and Senders

We need:

- Receiver for Q1
- Sender for Q2
- Sender for Q3

Example:

```
ServiceBusReceiver receiver =
    client.CreateReceiver("Q1");

ServiceBusSender senderQ2 =
    client.CreateSender("Q2");

ServiceBusSender senderQ3 =
    client.CreateSender("Q3");
```

The architecture is:

```
Receiver
   ↓
  Q1

Sender Q2
   ↓
  Q2

Sender Q3
   ↓
  Q3
```

---

8. Receive the Message from Q1

First, receive the message from Q1.

```
ServiceBusReceivedMessage message =
    await receiver.ReceiveMessageAsync();
```

At this point, the message is received from Q1.

If using the default **Peek-Lock** mode, the message is locked but not permanently removed yet.

This is important because we want to complete it only after the transaction succeeds.

---

9. Create Transaction Scope

We can use `TransactionScope` to group the Service Bus operations into a single transaction.

```
using var transaction =
    new TransactionScope(
        TransactionScopeOption.Required,
        TransactionScopeAsyncFlowOption.Enabled);
```

The important option for asynchronous code is:

```
TransactionScopeAsyncFlowOption.Enabled
```

This allows the transaction context to flow correctly across asynchronous operations using `async` / `await`.

---

10. Send Message to Q2

Inside the transaction, send the received message to Q2.

```
await senderQ2.SendMessageAsync(
    new ServiceBusMessage(message.Body));
```

Conceptually:

```
Q1
 ↓
Receive
 ↓
Transaction
 ↓
Q2
```

---

11. Send Message to Q3

Next, send the same message to Q3.

```
await senderQ3.SendMessageAsync(
    new ServiceBusMessage(message.Body));
```

Now the transaction contains:

```
Q1 → Receive
Q2 → Send
Q3 → Send
```

---

12. Complete the Message from Q1

Only after successfully sending the message to Q2 and Q3 should we complete the original message from Q1.

```
await receiver.CompleteMessageAsync(message);
```

Now the transaction contains all three operations:

```
1. Receive from Q1
2. Send to Q2
3. Send to Q3
4. Complete message from Q1
```

---

13. Commit the Transaction

After all operations succeed:

```
transaction.Complete();
```

This tells the transaction that all operations have completed successfully.

Conceptually:

```
Q1 → Complete ✓
Q2 → Send ✓
Q3 → Send ✓
       ↓
Transaction.Complete()
       ↓
Commit
```

---

14. Complete Example

A simplified example is:

```
using System.Transactions;

ServiceBusReceivedMessage message =
    await receiver.ReceiveMessageAsync();

using var transaction =
    new TransactionScope(
        TransactionScopeOption.Required,
        TransactionScopeAsyncFlowOption.Enabled);

await senderQ2.SendMessageAsync(
    new ServiceBusMessage(message.Body));

await senderQ3.SendMessageAsync(
    new ServiceBusMessage(message.Body));

await receiver.CompleteMessageAsync(message);

transaction.Complete();
```

The important thing is that all Service Bus operations are executed inside the transaction scope.

---

15. What Happens If Everything Succeeds?

Suppose:

```
Q1 → Hello Transaction
Q2 → Empty
Q3 → Empty
```

Transaction starts:

```
Q1 → Receive
```

Then:

```
Q2 → Hello Transaction ✓
Q3 → Hello Transaction ✓
Q1 → Complete ✓
```

Finally:

```
transaction.Complete()
```

Result:

```
Q1 → 0 messages
Q2 → 1 message
Q3 → 1 message
```

Everything succeeds.

---

16. What Happens If Q3 Fails?

Suppose:

```
Q1 → Hello Transaction
Q2 → Empty
Q3 → Empty
```

Transaction starts:

```
Q1 → Receive ✓
Q2 → Send ✓
Q3 → Send ✗
```

Because Q3 failed, the transaction should not be committed.

The transaction is rolled back.

Conceptually:

```
Q1 → Message remains available
Q2 → Send is rolled back
Q3 → Send failed
```

Therefore, we avoid a partially completed operation.

This is the major benefit of using a transaction.

---

17. Atomicity

Cross-entity transactions provide **atomic behavior**.

Atomicity means:

> Either all operations succeed, or the transaction is rolled back.

For example:

```
Q1 → Complete ✓
Q2 → Send ✓
Q3 → Send ✓
```

All succeed → **Commit**

But:

```
Q1 → Complete ✓
Q2 → Send ✓
Q3 → Send ✗
```

Not all operations succeed → **Rollback**

This prevents inconsistent results.

---

18. Cross-Entity Transaction vs Normal Operation

Without Transaction

```
Receive Q1
   ↓
Send Q2
   ↓
Send Q3
   ↓
Complete Q1
```

If something fails in the middle, the application may end up in an inconsistent state.

With Cross-Entity Transaction

```
Start Transaction
       ↓
Receive Q1
       ↓
Send Q2
       ↓
Send Q3
       ↓
Complete Q1
       ↓
Commit
```

If any operation fails:

```
Rollback
```

---

19. Important Requirements

When using cross-entity transactions, remember:

1. Enable the feature

```
EnableCrossEntityTransactions = true
```

2. Entities should belong to the same Service Bus namespace

For example:

```
Namespace
 ├── Q1
 ├── Q2
 └── Q3
```

3. Use a transaction scope

```
TransactionScope
```

4. Enable async flow

```
TransactionScopeAsyncFlowOption.Enabled
```

This is important when using `await`.

5. Complete the transaction

```
transaction.Complete();
```

---

20. Real-Time Use Case

Consider an order processing system.

An order message arrives in Q1:

```
Q1 = Order Processing
```

The application needs to send the order to:

```
Q2 = Payment Processing

Q3 = Inventory Processing
```

The desired flow is:

Only when both operations succeed should the original message in Q1 be completed.

This helps maintain consistency across multiple Service Bus entities.

---

21. Interview Questions

Q1. What is a cross-entity transaction in Azure Service Bus?

Answer:

A cross-entity transaction allows multiple operations involving different Service Bus entities, such as queues or topics, to be performed as one atomic transaction. If all operations succeed, the transaction is committed; otherwise, the operations are rolled back.

Q2. Why do we need cross-entity transactions?

Answer:

They are used when we need to perform operations on multiple Service Bus entities and want to ensure that either all operations succeed or none of them are committed.

For example:

```
Read from Q1
Send to Q2
Send to Q3
Complete Q1
```

All these operations can be performed as one transaction.

Q3. What property needs to be enabled for cross-entity transactions?

Answer:

```
EnableCrossEntityTransactions = true
```

This is configured in `ServiceBusClientOptions`.

Q4. Why do we use TransactionScopeAsyncFlowOption.Enabled?

Answer:

Because Service Bus operations are asynchronous and use `async`/`await`.
`TransactionScopeAsyncFlowOption.Enabled` allows the transaction context to flow correctly across asynchronous operations.

Q5. What happens if sending to Q3 fails?

Answer:

The transaction should not be committed. The transaction is rolled back, preventing the application from ending up with a partially completed set of operations.

---

22. Easy Way to Remember

Remember the example:

```
Q1 → Read
 ↓
Q2 → Write
 ↓
Q3 → Write
 ↓
Q1 → Complete
 ↓
COMMIT
```

One-Line Definition

> **Cross-entity transaction = Multiple Azure Service Bus operations across entities treated as one atomic unit.**

Interview Shortcut

"All succeed → Commit. Any failure → Rollback."

---

23. Key Points for Revision

- Cross-entity transactions allow operations across multiple Service Bus entities.
- Example: Read from Q1 and send to Q2 and Q3.
- The original message should be completed only after the destination operations succeed.
- Enable:

```
EnableCrossEntityTransactions = true
```

- Use `TransactionScope`.
- For async operations, use:

```
TransactionScopeAsyncFlowOption.Enabled
```

- Call:

```
transaction.Complete();
```

to indicate successful completion of the transaction.

- If the transaction cannot complete successfully, the operations are rolled back.
- Cross-entity transactions help maintain **consistency and atomicity** across Service Bus entities.

Final Flow